Using UPPAAL to Model and Verify a Clock Synchronization Protocol for the Controller Area Network

Guillermo Rodríguez-Navas, Julián Proenza
Systems, Robotics and Vision Group
Departament de Matemàtiques i Informàtica
Universitat de les Illes Balears, Spain
E-mail: guillermo.rodriguez-navas@uib.es, julian.proenza@uib.es

Hans Hansson
MRTC
Department of Computer Science and Electronics
Mälardalen University, Sweden
E-mail: hans.hansson@mdh.se

***Abstract*— A reported liability of the Controller Area Network protocol is that it does not provide a clock synchronization service. Therefore, whenever a CAN-based distributed embedded system requires its nodes to have a common time base, clock synchronization has to be implemented by means of an external mechanism. In a previous work, we proposed a fault-tolerant and high-precision clock synchronization protocol for CAN. This paper shows the first steps towards the formal verification of this protocol. In particular, it presents a formal model that has been built with the UPPAAL model checker and discusses how clock drift and clock correction can be modeled with this tool.**

I. Introduction

The Controller Area Network (CAN) field bus [1] is nowadays extensively used for the implementation of distributed embedded systems. In fact, although it was initially designed for in-vehicle communications, it has been adopted in a broad range of applications, such as factory automation, medical equipment and building automation, among others. One reported liability of the Controller Area Network protocol is its lack of a clock synchronization service [2]. Due to this, this service is usually implemented by means of an external mechanism, typically a software application executed by the nodes.

The existence of a system-wide synchronized clock is frequently assumed when designing distributed applications, and hence mechanisms are designed which rely on this assumption. For these mechanisms to work properly it is very important to guarantee that the clock synchronization service is reliable enough. In a previous work, some of the authors proposed a solution for clock synchronization in CAN networks [3], which is intended to provide the system with a clock of good precision despite the occurrence of a wide range of channel and nodes' faults.

This paper presents the first steps towards the formal verification of this protocol by means of *model checking* [4]. The UPPAAL model checker [5] has been chosen for this verification as it has been successfully used to verify other real-time communication protocols, e.g. in [6], [7]. In particular, this paper includes a thorough description of the formal model that has been built to specify our clock synchronization protocol. An additional contribution of this work is to show how clocks of variable drift can be modeled with UPPAAL. To the authors' best knowledge, this has not been described before.

The rest of the paper is organized as follows. Section II introduces the basic features of the UPPAAL model checker. In Section III, the system model is presented and discussed, whereas the fault model is presented in Section IV. Section V is devoted to describing the main characteristics of our clock synchronization protocol. Section VI focuses on the description of the verification model, paying special attention to the mechanisms that we have proposed to model clock drift and clock correction. Finally, Section VII summarizes the paper and gives some insight for further improvement.

II. UPPAAL background

UPPAAL is a model checker that allows definition of the system behavior by means of a network of timed automata. A *timed automaton* is a finite state machine extended with clock variables, which are used to measure time progress. The theory of timed automata allows clocks to be evaluated to a real number, but UPPAAL restricts those evaluations to integer numbers.

An UPPAAL timed automaton is made up of a number of *locations* and a number of *transitions* (also known as *edges*) between them. A transition usually causes an update of the system's variables. In particular, as a consequence of a transition, integer and boolean variables can be assigned a new value, whereas clock variables can only be restarted.

Synchronous actions (i.e. actions that are simultaneously performed by several automata) can be modeled by means of *synchronous channels*. Such transitions lead to a new state of the system in which the involved automata may all have stepped into a new location. A transition (either synchronous or not) is enabled as long as its corresponding *guard condition* holds. Guard conditions can use integer, boolean and clock variables, yet with some restrictions.

There is a particular kind of locations, the so-called *committed locations*, which help to model atomic actions. These loca-

0-7803-9402-X/05/$20.00 © 2005 IEEE

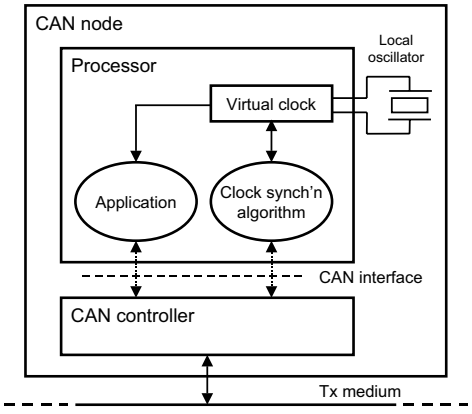

Fig. 1. Structure of a CAN node

tions are always left immediately and can interleave only with other commited locations. Additionally, UPPAAL incorporates three mechanisms to model urgency, namely *invariants*, *urgent channels* and *urgent locations*, making it possible to force certain transitions to be fired as soon as they are enabled. Whenever these mechanisms are not used, a transition may take place at an undetermined instant.

A fundamental characteristic of a network of timed automata is that clocks progress at the same rate. Nevertheless, in this work some techniques have been developed which allow us to model clocks of different rates as well. These techniques are described in Section VI.

III. System model

This paper addresses the problem of clock synchronization in CAN-based distributed embedded systems. Such systems are composed by a number of so-called CAN nodes, which execute a coordinated function and use a broadcast network to exchange messages. In this work it is also assumed that the CAN nodes require a common time base in order to carry out such coordinated function.

A. Node structure

The structure of a CAN node is depicted in Figure 1. The CAN node is made up of two basic elements: a *processor*, which executes both the application software and the clock synchronization algorithm and a *CAN controller*, which provides the processor with the CAN communication capabilities. The processor and the CAN controller communicate through the so-called *CAN interface*.

Note that the processor is clocked by a local oscillator. This local oscillator is used to measure time, by means of a software counter which simply acts as a frequency divider. This software-implemented counter is called the *virtual clock*.

In order to guarantee that the virtual clocks of the various CAN nodes do not drift apart, each node executes a clock synchronization algorithm. The function of this algorithm is to compare the value of the local virtual clock with a valid reference and, if needed, make the appropriate corrections to the virtual clock. The valid reference is obtained through the CAN network.

B. CAN interface

The interface of the CAN controller includes four primitives to handle message transmission and reception, namely *Tx.request*, *Tx.confirm*, *Rx.indication* and *Abort.request*. The first three primitives are supported by any commercial CAN controller. The last primitive is not part of the CAN standard but it is supported by many available CAN controllers [8].

The actual implementation of each primitive may vary from one CAN controller to another, but their function and required parameters are the following:

- The *Tx.request(msg_id, msg_data)* primitive is used by the processor to request the transmission of message *msg_id*. This transmission may not take place immediately as it depends on the channel conditions.
- The *Tx.confirm(msg_id)* primitive is used by the CAN controller to indicate that message *msg_id*, whose transmission was previously requested, has been actually transmitted.
- The *Rx.indication(msg_id, msg_data)* primitive is used by the CAN controller to indicate that message *msg_id* has been received.
- The *Abort.request(msg_id)* primitive allows the processor to request the CAN controller to abort the transmission of the message *msg_id* (whose transmission was requested previously by the processor itself). It is important to remark that when a message transmission has started it cannot be aborted.

C. Relevant properties of the CAN controller

The clock synchronization algorithm that is presented in this paper relies on a number of properties that are to be provided by the CAN controller:

(i) **Prioritized medium access**. The CAN protocol specification defines a distributed bitwise algorithm to arbitrate the bus access. It guarantees that, whenever two or more nodes attempt to simultaneously send a frame, the node that succeeds is the one that sends the frame with the highest priority (i.e. the lowest identifier).

(ii) **Bounded response time**. Thanks to the priority-based access to the medium, and as long as there is a bounded number of channel errors, the worst-case response time of a CAN message is bounded and can be calculated [9], [10].

(iii) **Inconsistent broadcast**. In a CAN network, any transmitted message is expected to be received by all of the non-faulty nodes in the system. However, it has been reported that certain fault scenarios exist in which this property does not hold [11], [12]. The consequences of these scenarios can be either inconsistent message omissions or inconsistent message duplicates, as defined in [11].

(iv) **Start of frame signaling**. Our solution for clock synchronization requires the CAN controller to be able to

signal the first bit (the *Start of Frame* bit) of any frame so the virtual clock can be accurately sampled at that precise instant. Similar mechanisms have been suggested in the literature [13], [14] to improve the precision of the clock synchronization.

(v) **On-the-fly message timestamp**. The CAN controller must be able to overwrite the data field of a frame while it is being transmitted. In this way, the data field of a frame can convey a timestamp that indicates the value that the virtual clock of the transmitter exhibited when the *Start of Frame* bit of such frame was signaled.

It is important to remark that Properties (i) to (iii) are part of the standard CAN specification and thus they are supported by commercial CAN controllers. The mechanisms to provide Properties (iv) and (v) can be implemented with the help of programmable logic devices (PLDs). As explained in [3], we have used a Field-Programmable Gate Array (FPGA) to implement a modified CAN controller which fulfills both properties.

IV. Fault model

The system may suffer from three types of faults: software faults, physical faults of the nodes and physical faults of the channel. Due to the simplicity of our clock synchronization protocol, software faults are not included in the fault model.

Concerning physical faults of the nodes, the fault model includes arbitrary faults that can be either transient or permanent. However, an important assumption is that there is at least one non-faulty node in the system which can provide a clock of good quality.

Permanent physical faults of the channel, such as bus partition or stuck-at-dominant, are not included in the fault model. These faults can be addressed independently, for instance as proposed in [15].

Regarding transient channel faults, our fault model only includes those channel faults that lead to frame errors detectable by the error-detection mechanisms of CAN, since the probability of having a channel fault which is not detected by the CAN controllers is very low and can be neglected [16].

It is also assumed that the number of channel faults within a given time interval is bounded. In this way, it is possible to assume a bounded response time for every CAN message. Moreover, although our fault model includes channel faults that can lead to inconsistent frame receptions, the probability of such faults is low enough so as to assume that only a bounded number of consecutive resynchronization rounds can be affected by these faults.

V. Protocol description

In this section the basics of our proposal for clock synchronization in CAN networks are described.

A. Protocol basics

Our protocol adopts a master/slave approach, as there is one node (the master) that imposes its time view to the rest of nodes of the system (the slaves). The master is assumed to be synchronized to an external time source which supplies the desired accuracy. The master's time view is spread throughout the network by means of a specific message: the Time Message (TM), which the master periodically broadcasts.

The aim of the Time Message is twofold. On the one hand, the master uses it to indicate the resynchronization event, which is the sample point of the *Start of Frame* bit of this particular message. On the grounds of Property (iv), every slave samples its virtual clock at this instant and keeps the value in a variable called lt_ref. On the other hand, thanks to Property (v), the TM's data field contains the timestamp that the master took precisely at that instant, so each slave reads the TM and keeps the master's timestamp in a variable called rt_ref.

In this way, after reception of the TM each slave compares its virtual clock to the master's one and makes the required corrections. In particular, two corrections are carried out. One aims at correcting the *offset error*, which is calculated according to Equation 1, whereas the other aims at correcting the *drift error*, which is caused by the rate difference between the local virtual clock and the remote virtual clock, as shown in Equation 2.

$$offset\_error = rt\_ref - lt\_ref \tag{1}$$

$$drift\_error = \frac{local\_rate}{remote\_rate} = \frac{lt\_ref - rt\_ref_{prev}}{rt\_ref - rt\_ref_{prev}} \tag{2}$$

These corrections are not performed instantly, but they are progressively carried out [3]. This is often called *clock amortization* and is more advisable than instant corrections because it does not cause time leaps. Both offset correction and drift correction are never perfectly performed, since there is always some residual error caused by small system latencies or by the imprecision of the arithmetical operations. However, in our system the residual error of the offset correction is small, mainly because of Property (iv), and can be neglected. In contrast, the drift error (though being small) causes clocks to drift apart as time passes by, so it may have greater effect on the clock synchronization protocol and cannot be neglected. The residual drift error is denoted γ_{min} hereafter. The maximum drift error between any pair of slaves that have synchronized to the same master is $2 \times \gamma_{min}$.

The instant at which the master attempts to transmit the TM is called the *release time*. Even though the release time is periodical, the instant at which the TM is actually broadcast may vary depending on the network conditions. For instance, channel errors may cause frame retransmissions and therefore a significant delay.

Despite its simplicity, our protocol has several advantages. Thanks to the master/slave approach it is not expensive in terms of bandwidth consumption nor processor utilization. Due to this, and in contrast to solutions previously suggested [17], it adapts very well to the asymmetric nature of many distributed embedded systems, where nodes with bad quality clocks as well as scarce processing capabilities may exist. A

master/slave approach has been also adopted by the IEEE 1588 standard for clock synchronization in distributed systems [18].

B. Master replication

The main drawback of adopting a master/slave scheme is that the master represents a single point of failure since any failure of this node might cause a failure of the whole clock synchronization service. In our proposal, this single point of failure is eliminated by means of *master replication*. Basically, a number of master replicas or *backup masters* are defined, which supervise the so-called *active* master so one of them can take over in case this master is faulty.

In order to simplify the management of the master replication, the failure semantics of the masters (either active or backup) has been restricted to *crash failure semantics*, which means that upon a physical fault the node becomes silent. This failure semantics can be enforced by means of internal duplication and comparison, as explained in [3].

Owing to the restricted failure semantics of the masters, the faulty master detection and replacement mechanism is implemented by just defining the release time of the backup masters a short time after the release time of the active master. This short delay is different at each backup master, and is greater for those backup masters with lower priority. In this way, if the active master was faulty and did not broadcast its TM, the system would recover a short time later, as soon as the next backup master succeeded in broadcasting its own TM. This prioritized mechanism for master replacement implicitly assumes that masters' clocks are synchronized to a given precision Π. In particular, letting Γ be the separation between master i and master $i+1$, it assumes that $\Pi \leq \frac{\Gamma}{2}$.

Since channel conditions can delay the transmission of a TM, it is possible for one or more backup masters to reach their release time even if the active master is not actually faulty. Due to this, the TM identifiers are assigned in such a way that in case more than one masters attempt to send the TM simultaneously, the TM with highest priority is always broadcast first. After reception of a higher priority TM, each lower priority master requests its own CAN controller to abort the TM transmission. Nevertheless, the abort operation has some latency and it is not likely to be performed before the corresponding CAN controller starts the transmission of the following TM.

This implies that it is possible to receive two TMs from different masters within the same resynchronization round. In case of a consistent broadcast this fact would not be a problem because nodes only resynchronize to the first TM they receive. But in case of an inconsistent broadcast this might cause some nodes to resynchronize to a master whereas the rest resynchronize to another one. The consequences of these inconsistency scenarios are further explained in Section V-C

A drawback of enforcing crash failure semantics in the masters by means of internal duplication and comparison is that any transient fault turns into a permanent failure of the master, and this may cause fast *redundancy attrition*. This problem could be overcome by providing the masters with some internal check and recovery mechanisms, and by designing a proper policy for master reintegration. This issue has not been addressed in this work, but it seems that a suitable policy would be to allow any just-recovered master to reintegrate firstly as a slave (a sort of *passive* mode) and then, after a few correct resynchronization rounds, allow it to act again as a backup master.

C. Scenarios of inconsistent clock resynchronization

As it was stated in Section IV, certain channel faults may cause inconsistent omissions or inconsistent duplicates of the TM. Inconsistent TM duplicates do not have any impact on the clock precision because they do not cause any inconsistent resynchronization. In contrast, inconsistent omissions of the TM can have a negative impact on the clock synchronization, particularly when TMs from two different masters are broadcast. These potential inconsistency scenarios and their effect on the clock parameters are explained next:

- A backup master may not receive the TM sent by the active master nor any other TM of higher priority than its own. In such a case, the master would not correct its offset error with respect to the active master's clock. If the drift error of this backup master after the last successful resynchronization was γ then the accumulated offset error would be at most equal to γ times the elapsed time since that resynchronization. The drift error would be neither corrected, but the effect would not be too important since clock drift tend to change smoothly and therefore, for a time short enough, the drift error can be considered as being constant.
- A set of slaves may not receive the TM sent by the active master nor any other subsequent TM within one resynchronization round. The effect on the clock would be exactly the same as in the previous case as neither the offset error nor the drift error would be corrected.
- A set of nodes (including backup masters and slaves) may not receive the TM sent by the active master but do receive a subsequent TM from a backup master. In such a case, these nodes correct their clock according to that backup master, and *not* according to the active master. Due to this, they actually "inherit" both the offset error and the drift error of this backup master, which might be even greater than the errors these nodes had before resynchronization. Moreover, the drift error of these nodes may increase due to the residual drift error. For instance, if γ is the drift error (w.r.t. the active master) of the backup master that transmitted the second TM, then after resynchronization the drift error of the resynchronizing nodes may be $\gamma + \gamma_{min}$.

D. Protocol implementation

Figure 2 shows the clock synchronization algorithm as implemented by a generic master i. Every master (either active or backup) executes the same procedure, the only difference being the value of the release time. A Master can be in one of four states: Idle1, Queue, Abort and Idle2.

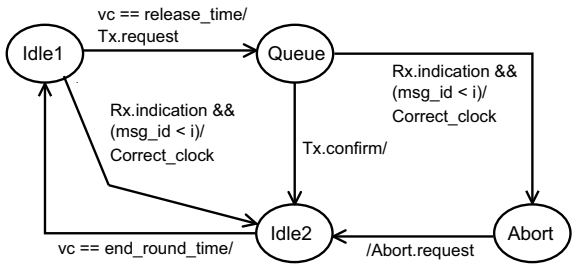

Fig. 2. Master i's finite state machine

Idle1 is the state at the beginning of the resynchronization round. In this state only two events may happen: either a TM is received or the $release\_time$ is reached. If a TM is received and it has higher priority than the master itself (a $Rx.indication$ is received with ($msg\_id < i$)) then master i performs the corresponding clock correction and steps into state Idle2. Idle2 represents the state at the end of the resynchronization round, when the master has already corrected its clock. While being in state Idle2, any other message reception is ignored.

Whenever master i is in state Idle1 and the $release\_time$ is reached (i.e. $vc == release\_time$), the master requests the transmission of its TM ($Tx.request$) and steps into state Queue. This means that master i is a candidate to become the active master. In state Queue only two events may happen: either the TM is successfully transmitted or a TM is received. If master i sends its TM (primitive $Tx.confirm$) before receiving any TM of higher priority then it considers itself as being the master of highest priority and steps into state Idle2 without performing any clock correction. In contrast, if master i receives a TM of higher priority ($Rx.indication$ with ($msg\_id < i$)) then it corrects its clock and moves to state Abort.

In state Abort, master i performs a request to abort the queued TM (primitive $Abort.request$) and proceeds into state Idle2 regardless of the result of the abort operation. As soon as the end_round_time time mark is reached (i.e. $vc == end\_round\_time$), Master i steps into state Idle1 and a new resynchronization round is started. The value of the end_round_time time mark is the same for all of the nodes.

The implementation of the slaves is not shown because it is very simple, and can moreover be derived easily from the master's state machine. Slaves only have two states (Idle1 and Idle2) and the two transitions between them that appear in Figure 2.

VI. UPPAAL VERIFICATION MODEL

This section describes the verification model that has been built in order to model our clock synchronization protocol. First, we discuss the techniques we have developed to model variable clock drift in UPPAAL. After that, an overview of the verification model is provided. Finally, and due to space limitations, only the main features of this model are discussed. A more detailed description of the model features can be found in [19].

A. How to model variable clock drift in UPPAAL

In order to understand the technique for modeling clock drift that has been used in this work it is necessary to examine the effects of having unsynchronized clocks in a distributed system. Imagine a set of nodes, in which every node is provided with a virtual clock and has to trigger an action when this virtual clock reaches the instant $t0$. These actions will be triggered simultaneously only in the ideal case of having perfectly synchronized clocks. In a realistic case, with clocks of slightly different rates, each node triggers the action at a different instant, i.e. those nodes with faster clocks trigger the action sooner than those with slower clocks. Moreover, this temporal uncertainty on the time-triggered actions is the only effect of having unsynchronized clocks. Note that temporal uncertainty is proportional to $t0$ since, in the absence of clock resynchronization, clocks tend to drift apart as time passes by.

Although it is impossible to determine the exact drift of a specific clock, it is possible to bound, up to a certain probability, the maximum drift that a clock may exhibit[1]. From this information, the *earliest* and the *latest* possible time of an event occurrence can be deduced. Using these extreme occurrence times the possible clock drifts can in UPPAAL be modeled as a time interval. This is illustrated in Figure 3, in which the transition from location L1 to location L2 can only happen if clock x is within the interval $[t0 - epsilon, t0 + epsilon]$.

By giving the appropriate values to $epsilon$ it is possible to model clock drifts, as the following example illustrates. Imagine that two automata (Master and Slave) exist which behave like the automaton in Figure 3, and that have restarted their local clocks x simultaneously. Master's clock is assumed to be the time reference whereas Slave's clock has a drift error equal to the residual drift error γ_{min}. Therefore, at instant $t0$ the value of $epsilon$ in the Slave automaton is equal to $t0 \times \gamma_{min}$ whereas the value of $epsilon$ in the Master is 0, as it does not drift with respect to itself.

Assuming these values, it can be deduced that automaton Slave may fire its transition from L1 to L2 within a time interval that is centered at $t0$ and has a length equal to $2 \times t0 \times \gamma_{min}$. Since the transition from L1 to L2 is used to restart each local clock x, the consequence of this uncertainty is that the offset difference between Master's clock and Slave's clock (when both Master and Slave are in location L2) varies in the range $[-t0 \times \gamma_{min}, t0 \times \gamma_{min}]$.

This mechanism can be adapted to our model because there are only two time-triggered actions, the broadcast of the TM and the end of the round, and both are periodical. Therefore, it is possible to determine *a priori* when they are going to take place, and hence, $epsilon$ can be calculated as long as the drift error (γ) of each clock is known. Since the value of γ changes

[1]In our system, this assumption is also enforced by the duplication with comparison structure of the masters as long as two clocks are used

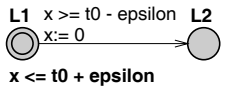

Fig. 3. Example of automaton

according to the resynchronization actions that are performed, it has to be calculated after every resynchronization in the way that is described in Section VI-C.

It is important to remark that by modeling clock drift in this manner, not only the extreme drift values are considered, but also all of the intermediate drifts. This represents a great advantage in front of simulation techniques, in which only a finite set of drifts can be evaluated. A similar approach, yet not applied to clock drift, has been proposed to model hybrid systems with timed automata [20].

B. Model overview

The model presented in this paper does not include any automaton to model the slaves' behavior. This means that only clock resynchronization among masters can be verified. This is due to the fact that it was decided that masters' behavior had to be addressed first as it is the most complex part of the clock synchronization protocol. Once masters' resynchronization has been modeled and verified, it seems that slaves can be easily incorporated into the model. In fact, and according to what is said in Section V-D, the slave automaton is a subset of the master's one.

Our clock synchronization model includes the following automata:

- One *Master* automaton (Figure 4) for each master in the system. This automaton models the clock resynchronization protocol itself.
- One *Clk_ctrl* automaton (Figure 5) for each master in the system. This automaton helps to model clocks of variable drift.
- One *Chan_ctrl* automaton (Figure 6), which models part of the CAN communication.
- One *Round_ctrl* automaton (Figure 7), which is used mainly to restart the system variables at the end of each resynchronization round.

The automaton in Figure 4 is actually a simplified model of the Master automaton since, for the sake of clarity, the following features have been deliberately omitted: inconsistent reception of messages, the possibility of having TMs whose transmission is not timely aborted, and master crash. The way to add all of these features to our model is described in [19].

Notice that the simplified Master automaton includes a location for each of the states that appear in Figure 2, though a number of additional locations have been incorporated in order to model the interaction with the rest of the system.

Our model includes N masters. Each master is given a unique identifier my_id in the range $[0, N-1]$. The identifier is used to indicate the priority of the master and therefore the priority of the TM it broadcasts. A lower identifier means higher priority, which is the usual convention in CAN networks.

Furthermore, each master is provided with its own virtual clock. This is modeled with an array of clocks vc of size N, such that $vc[my\_id]$ refers to the virtual clock of the my_id-est master. The array vc is a global variable, so every master can read the virtual clock of any other master. This is very useful in order to model clock correction.

The model also includes a number of constants and variables that are related to the clock correction procedure. On the one hand, the array of integers $gamma$, which has size N, keeps the current drift error of each master (γ). This drift error is measured with respect to the active master, so $gamma[my\_id]$ equals to 0 only if Master my_id is the current active master. The value of $gamma[my\_id]$ is calculated right after every clock correction, and is always an integer multiple of γ_{min} (constant $gamma\_min$ in the model). Due to this, and in order to reduce the memory requirements of the model, only the integer value is kept. On the other hand, the constant $gamma\_step$ represents the effect that a drift equal to γ_{min} causes after $R1 + delta$ time units. It is calculated as follows: $gamma\_step = (R1 + delta) \times gamma\_min$, so it has a different value for each master.

C. Main model features

These are the main features our model includes.

1) Uncertainty of the release time: As said in Section V-B, each master has a different release time, which depends on the master's priority. In the model, each Master automaton uses a local integer variable $delta$ to represent its delay with respect to the release time of Master 0. The release time of Master 0 is $R1$ time units after the end of the last resynchronization round, so every master reaches its release time when its virtual clock measures $R1 + delta$ time units.

However, due to the drift with respect to the active master's clock, the release time of a master can occur within a time interval, whose length depends on the current drift error. As shown in Figure 4, this time interval is centered at $R1 + delta[my\_id]$ and its boundaries are proportional to $gamma[my\_id]$, which represents the drift error between clock $vc[my\_id]$ and the active master's virtual clock after the last resynchronization. The constant $gamma\_step$ is proportional to $R1 + delta[my\_id]$ and models the fact that time uncertainty increases for larger time values.

Since the value of $gamma[my\_id]$ is calculated in each resynchronization round, the boundaries of the release time interval may change in every round, depending on the actions performed by the nodes. This is a very important property of the model as it allows us to study how the resynchronization actions affect the node's clock drift and vice versa. The way of calculating the value of $gamma[my\_id]$ is explained later on in this section.

2) Prioritized medium access: This property is modeled by means of a global integer variable called msg_id. This variable indicates the identifier of the highest priority TM among all the TMs that are waiting for transmission. A msg_id value

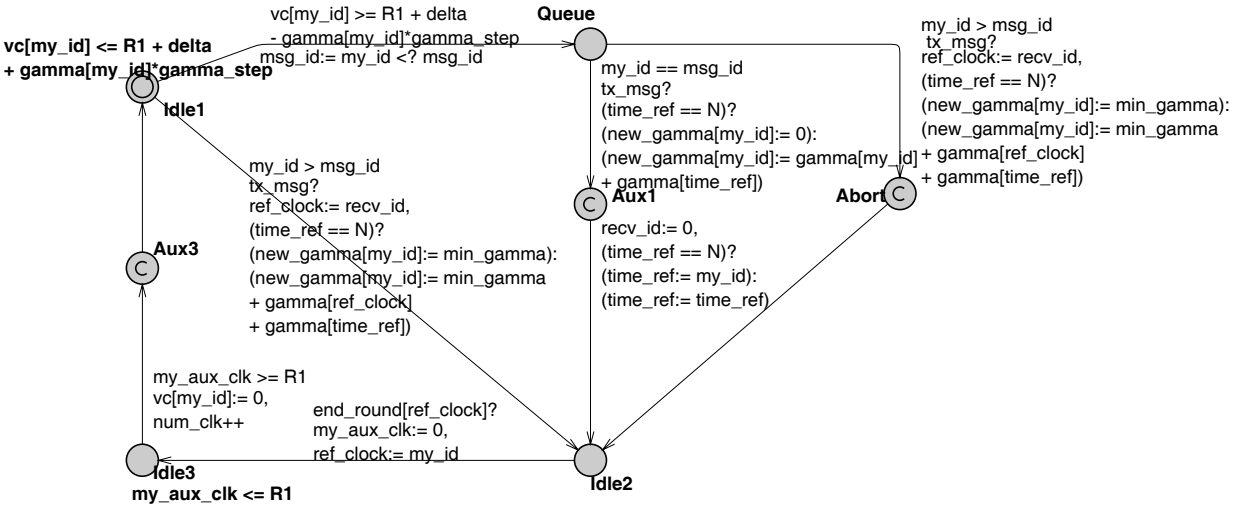

Fig. 4. Simplified Master automaton

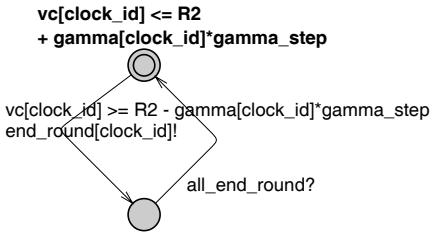

Fig. 5. Clk_ctrl automaton

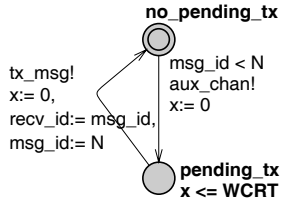

Fig. 6. Chan_ctrl automaton

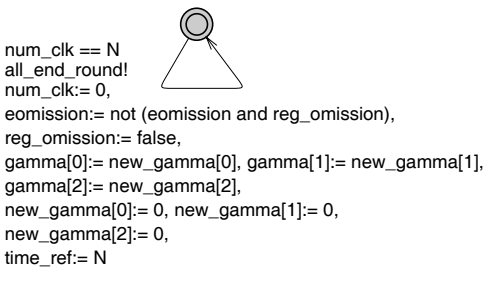

Fig. 7. Round_ctrl automaton

equal to N means that no message is waiting for transmission (the bus is idle).

Whenever a master wants to broadcast its TM (transition from Idle1 to Queue), it checks the current value of msg_id, and only if its own identifier (local integer variable my_id) is lower than msg_id then it overwrites msg_id with its identifier (the C-like assignment $msg\_id := my\_id <?msg\_id$ models this behavior). In this way, if a master of higher priority had previously requested a TM transmission then the master could not overwrite the variable msg_id, which in fact would be equivalent to losing the arbitration.

3) Clock correction: The set of Clk_ctrl automata are intended to signal the end of the round, as seen by the various masters in the system. Every Clk_ctrl automaton is identified by its own $clock\_id$, so Clk_ctrl automaton indicates the end of the round as measured by the virtual clock of Master $clock\_id$. This is signaled through the broadcast channel $end\_round[clock\_id]$. This event can occur within a time interval the length of which depends on both the drift of Master $clock\_id$ (kept in $gamma[clock\_id]$) and the variable $gamma\_step[clock\_id]$.

Moreover, every Master automaton is provided with a local integer variable called ref_clock, which is used in the following way to model clock correction. Whenever one master resynchronizes to a received TM, the master makes $ref\_clock := recv\_id$. After that, this master does not use its own clock to measure the end of the round, but the virtual clock of the master to which it has resynchronized. This is modeled with the broadcast channel $end\_round[ref\_clock]$. Thanks to this, all of the masters that resynchronized to the active master leave location Idle2 simultaneously. In contrast, and due to what was explained above about the Clk_ctrl automaton, those masters that resynchronized to a different master may leave location Idle2 sooner or later than the active master.

When leaving location Idle2, each master resets a local

clock aux_clock. As soon as this clock reaches R1 time units, the local virtual clock of the master $(vc[my\_id])$ is reset. In this way, the offset error is corrected only in those masters that resynchronized to the active master, whereas the rest of masters keep their offset for the next resynchronization round. Note that this is coherent to what is explained in Section V-C.

The new drift error of each master is calculated whenever one of the transitions leading to Idle2 that imply clock correction is taken. It is calculated taking into account whether the master resynchronizes to the active master (which is kwown thanks to variable $time\_ref$) or not. The new drift value is kept by the variable $new\_gamma[my\_id]$ until the end of the round. At the end of the resynchronization round, automaton Round_ctrl assigns the value of new_gamma to $gamma$, so the new drift error can be taken into account for the next resynchronization round.

VII. Conclusions and future work

In a previous work, we have proposed a solution for clock synchronization over CAN. This paper describes the first steps towards the formal verification of this solution with the UPPAAL model checker. UPPAAL is very well suited for the formal verification of real-time systems as it includes the notion of time. However, this tool is based on the timed automata theory, which assumes already synchronized clocks, and is thereby usually applied to systems with this kind of clocks. Nevertheless, we have shown that UPPAAL can be used also to model systems with clocks of variable drift. This has been a significant contribution of this work.

Moreover, our verification model includes a number of common characteristics of the CAN communication, such as arbitration, bounded response time and broadcast. We believe that the artifacts presented in this paper to model these features with UPPAAL might help researchers that aim at formally verifying their own CAN-based protocols. Even though some of these artifacts are not thoroughly described in this paper, a more detailed description can be found in [19].

Our verification model has been successfully used to verify certain properties of our clock synchronization protocol. In particular, some results concerning the correctness of the faulty master detection and replacement mechanism have been obtained, but they have not been presented in this paper as they are still preliminary. From this experience, we believe that our UPPAAL model is an excellent starting point so as to understand the complex relationship that there exists between the fault conditions and the best achievable clock precision.

A few additional features could still be included into the model. One of them is to allow an unbounded number of channel errors within a resynchronization round. In this way, it would be possible to study the effect of large error bursts on the clock service. Furthermore, a similar model could be designed in which the on-the-fly timestamping is not assumed for the CAN controllers. This would require the transmission of two messages: one to indicate the resynchronization instant and another one to carry the timestamp of such event. Although eliminating this requirement would make our solution much more generic, it is not clear how inconsistent omissions may affect the achievable precision.

Acknowledgement

This work has been partially supported by the Spanish MCYT grant DPI2001-2311-C03-02, which is partially funded by the European Union FEDER programme.

References

[1] ISO, "ISO11898. Road vehicles - Interchange of digital information - Controller area network (CAN) for high-speed communication," 1993.

[2] M. Törngren, "A perspective to the Design of Distributed Real-time Control Applications based on CAN," *Proceedings of 2nd International CAN Conference, London-Heathrow, United Kingdom*, 1995.

[3] G. Rodríguez-Navas, J. Bosch, and J. Proenza, "Hardware Design of a High-precision and Fault-tolerant Clock Subsystem for CAN Networks," *Proceedings of the 5th IFAC International Conference on Fieldbus Systems and their Applications (FeT 2003), Aveiro, Portugal*, 2003.

[4] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. Cambridge, Massachusetts: The MIT Press, 2001.

[5] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on UPPAAL," in *Formal Methods for the Design of Real-Time Systems: 4th International School on Formal Methods for the Design of Computer, Communication, and Software Systems, SFM-RT 2004*, ser. LNCS, M. Bernardo and F. Corradini, Eds., no. 3185. Springer–Verlag, September 2004, pp. 200–236.

[6] H.Lönn and P. Pettersson, "Formal Verification of a TDMA Protocol Startup Mechanism," in *Proceedings of the Pacific Rim Int. Symposium on Fault-Tolerant Systems*, 1997, pp. 235–242.

[7] A. David and W. Yi, "Modelling and analysis of a commercial field bus protocol," *Proceedings of the 12th Euromicro Conference on Real Time Systems*, pp. 165–172, 2000.

[8] K. Etschberger, *Controller Area Network*. Weingarten: IXXAT Press, 2001.

[9] K. Tindell, A. Burns, and A. J. Wellings, "Calculating controller area network (CAN) message response time," *Control Engineering Practice*, vol. 3(8), pp. 1163–1169, 1995.

[10] I. Broster, A. Burns, and G. Rodríguez-Navas, "Probabilistic analysis of CAN with faults," *Proceedings of the 23rd Real-time Systems Symposium*, 2002.

[11] J. Rufino, P. Veríssimo, G. Arroz, C. Almeida, and L. Rodrigues, "Fault-tolerant broadcasts in CAN," *Digest of papers, The 28th IEEE International Symposium on Fault-Tolerant Computing, Munich, Germany*, 1998.

[12] J. Proenza and J. Miro-Julia, "MajorCAN: A modification to the Controller Area Network to achieve Atomic Broadcast," *IEEE Int. Workshop on Group Communication and Computations. Taipei, Taiwan*, 2000.

[13] K. Turski, "A global time system for CAN," *Proceedings of the 1st International CAN Conference, Mainz, Germany*, 1994.

[14] T. Führer, B. Müller, W. Dieterle, F. Hartwich, R. Hugel, M. Walther, and R. B. GmbH, "Time Triggered Communication on CAN," *Proceedings of the 7th Int. CAN Conference, Amsterdam, The Netherlands*, 2000.

[15] J. Rufino, P. Veríssimo, and G. Arroz, "A Columbus' egg idea for CAN media redundancy," *Digest of Papers, The 29th International Symposium on Fault-Tolerant Computing Systems*, 1999.

[16] J. Charzinski, "Performance of the error detection mechanisms in CAN," *Proceedings of the First International CAN Conference (ICC94),Mainz, Germany*, 1994.

[17] L. Rodrigues, M. Guimaraes, and J. Rufino, "Fault-tolerant Clock Synchronization in CAN," *Proceedings of the 19th IEEE Real-Time Systems Symposium, Madrid, Spain*, 1998.

[18] IEEE-1588, *Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems*. IEEE Instrumentation and Measurement Society, 2002.

[19] G. Rodríguez-Navas, J. Proenza, and H. Hansson, "An UPPAAL Model for Formal Verification of Clock Synchronization over CAN," Universitat de les Illes Balears, Tech. Rep. A-2-2005, 2005.

[20] A. Olivero, J. Sifakis, and S. Yovine, "Using abstractions for the verification of linear hybrid systems," in *Proceedings of the 6th Int. Conference on Computer Aided Verification*, ser. LNCS, no. 818. Springer–Verlag, June 1994.